

QUIZ: File distribution time

100 clients would like to download a popular file (50MB). Each client has 1Mbps download capacity and 0.5Mbps upload capacity.

- How much time would it take if they are downloading from a server in a client-server architecture, assuming the server supports 10 concurrent downloads?

A: $10 * (50M * 8 / 1Mbps) = 4000 \text{ secs}$

- How much time would it take if they form a BitTorrent P2P system, assuming perfect distribution? Approximation is fine.

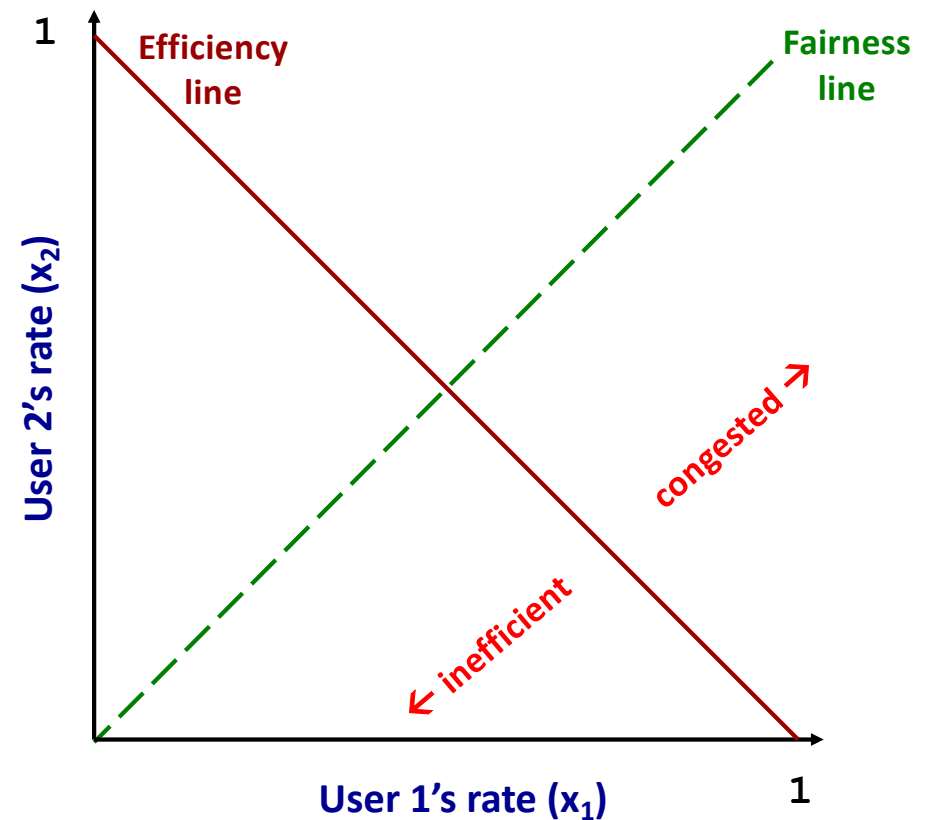
A: $50MB * 8 * 100 / (0.5Mbps * 100) = 800 \text{ secs}$

Other options?

- Every RTT, we can
 - **Multiplicative** increase or decrease: $CWND \rightarrow a * CWND$
 - **Additive** increase or decrease: $CWND \rightarrow CWND + b$
- Four options
 - AIAD: gentle increase, gentle decrease
 - AIMD: gentle increase, drastic decrease
 - MIAD: drastic increase, gentle decrease
 - MIMD: drastic increase and decrease

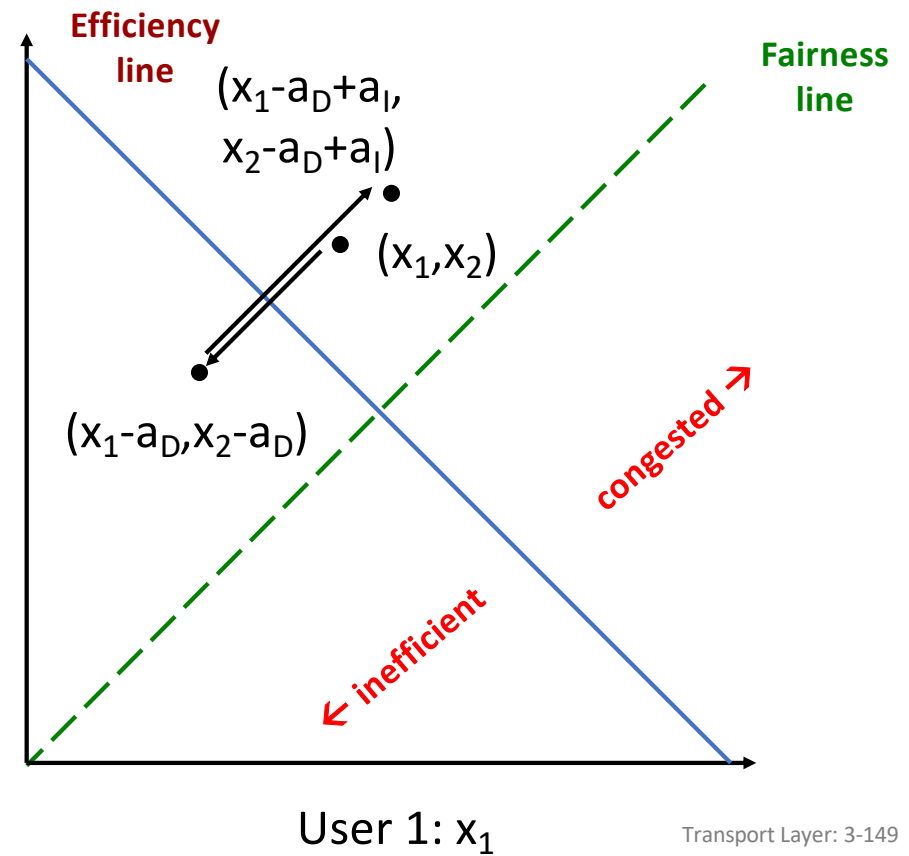
Simple Model of Congestion Control

- Two users
 - rates x_1 and x_2
- Congestion when $x_1 + x_2 > 1$
- Unused capacity when $x_1 + x_2 < 1$
- Fair when $x_1 = x_2$



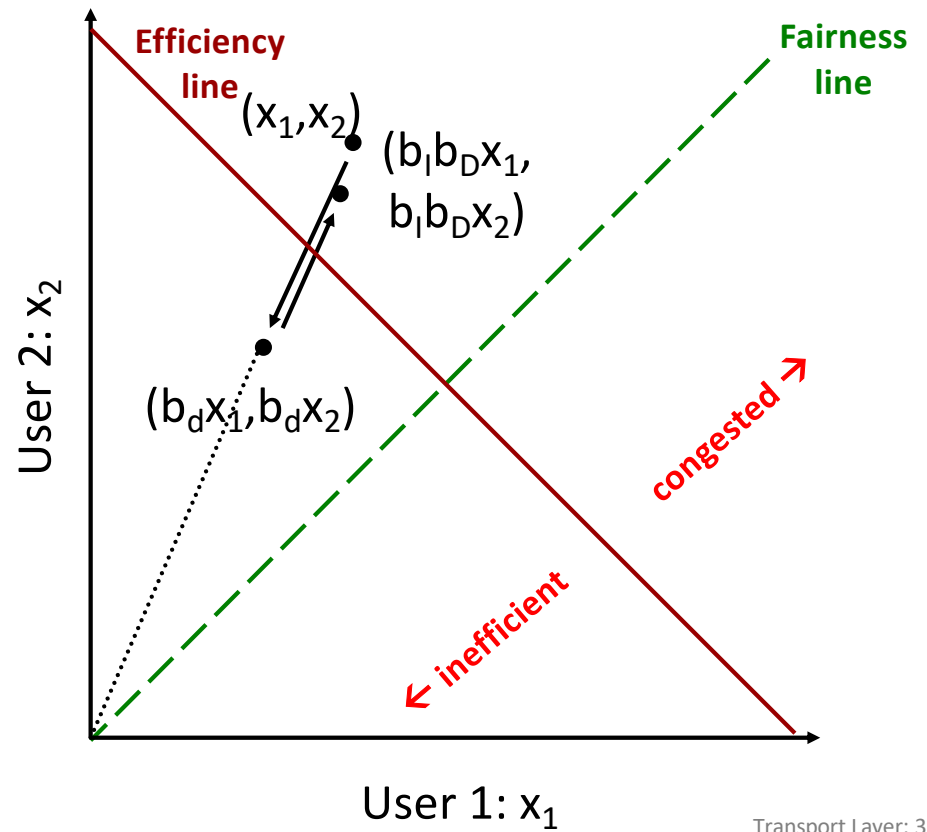
AIAD

- Increase: $x + a_I$
- Decrease: $x - a_D$
- **Does not converge to fairness**



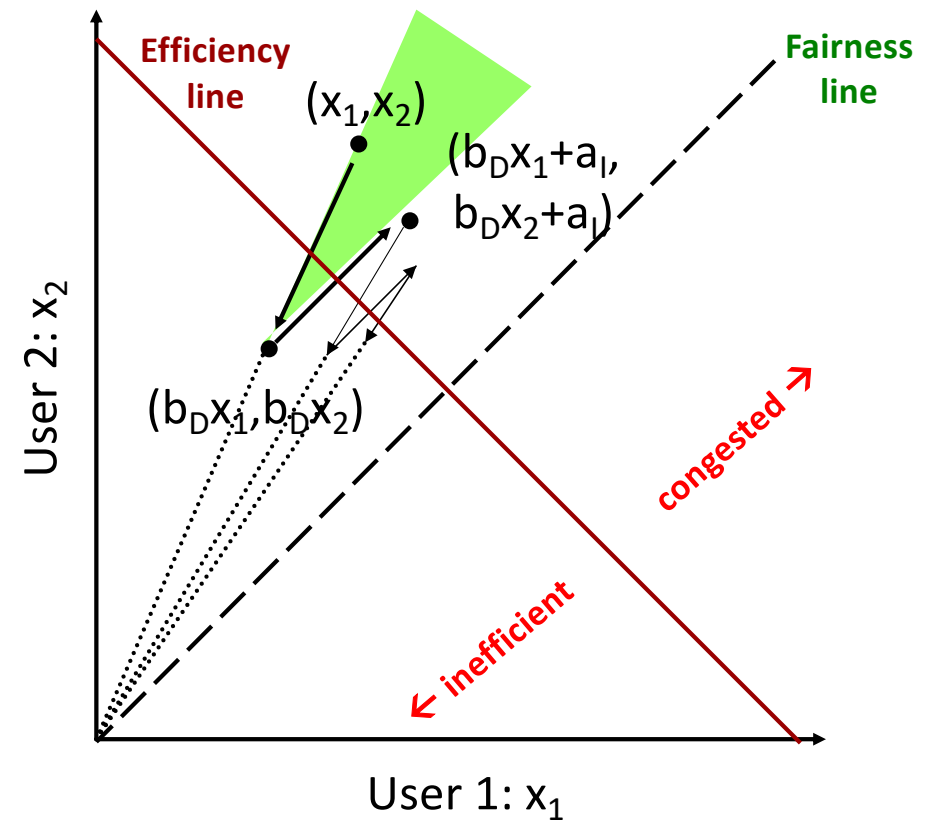
MIMD

- Increase: x^*b_I
- Decrease: x^*b_D
- **Does not converge to fairness**

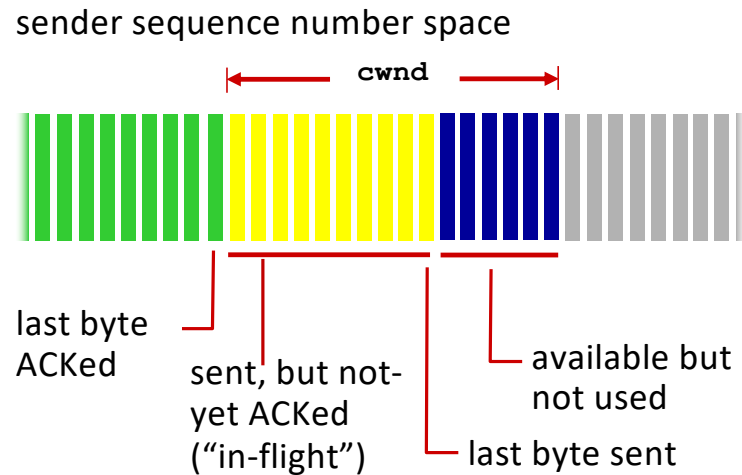


AIMD

- Increase: $x + a_l$
- Decrease: $x * b_D$
- **Converges to fairness!**



TCP congestion control: details



TCP sending behavior:

- *roughly*: send `cwnd` bytes, wait RTT for ACKS, then send more bytes

$$\text{TCP rate} \approx \frac{\text{cwnd}}{\text{RTT}} \text{ bytes/sec}$$

- TCP sender limits transmission: $\text{LastByteSent} - \text{LastByteAcked} \leq \text{cwnd}$
- `cwnd` is dynamically adjusted in response to observed network congestion (implementing TCP congestion control)

Two kinds of loss in TCP

- Triple duplicate ACK

- Packet n is lost, but packets $n+1$, $n+2$, ... arrive
- Receiver sends duplicate ACKs, sender resends packet n

→ Do a multiplicative decrease and keep going

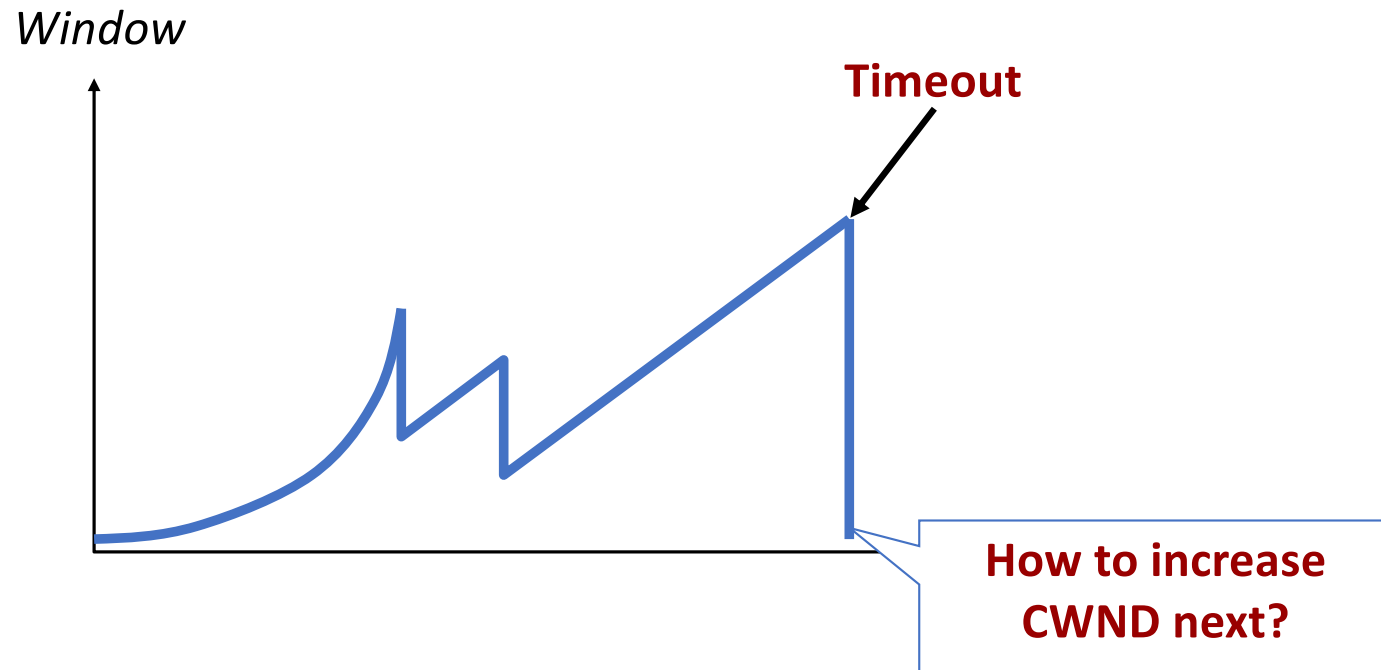
- Timeout

- Packet n is lost and detected via a timeout (e.g., because all packets in flight were lost)
- After the timeout, blasting away for the entire CWND, would trigger a very large burst in traffic

→ Better to start over with a low CWND

When timeout happens

- CWND goes back to one packet (1 MSS)



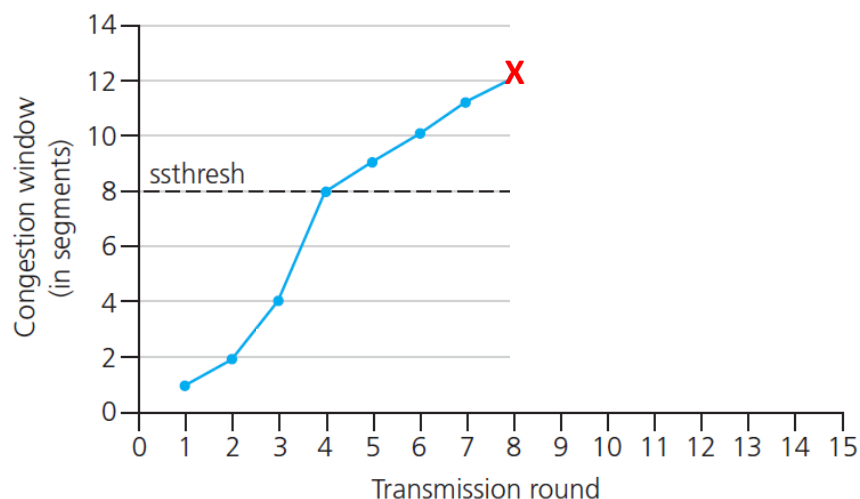
TCP: from slow start to congestion avoidance

Q: when should the exponential increase switch to linear?

A: when **cwnd** gets to 1/2 of its value before timeout.

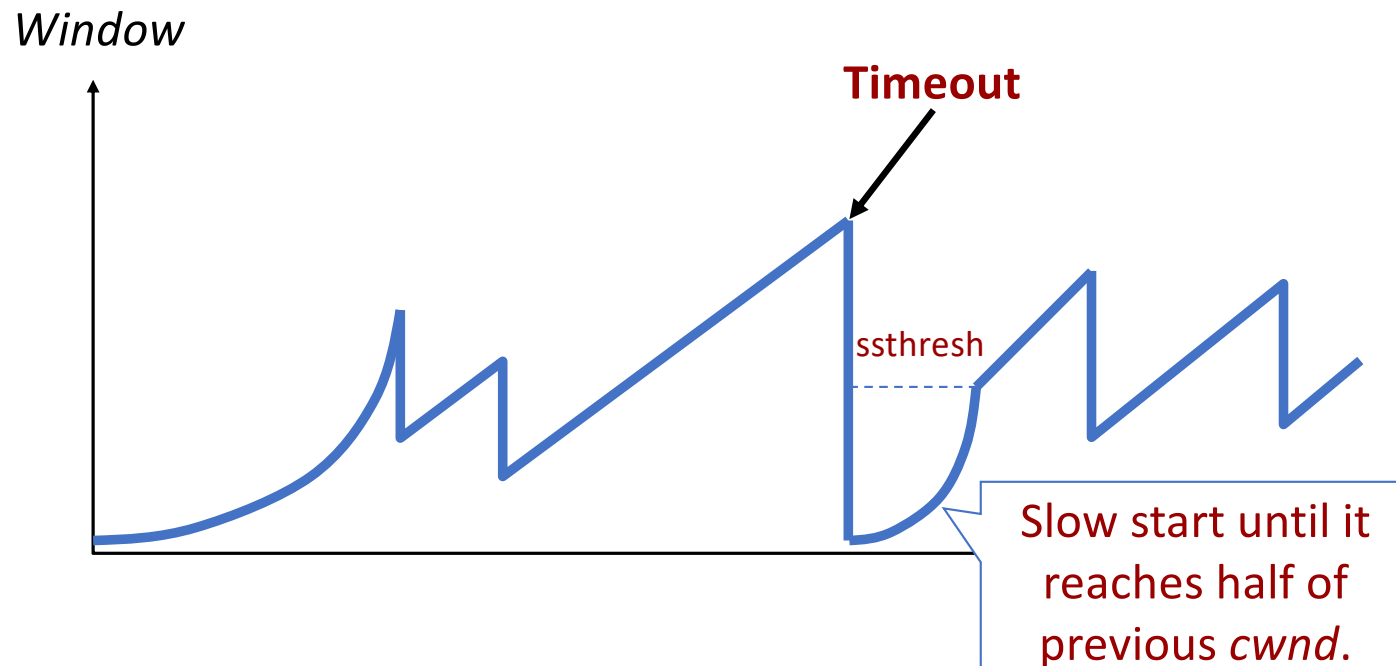
Implementation:

- variable **ssthresh**
- on loss event, **ssthresh** is set to 1/2 of **cwnd** just before loss event



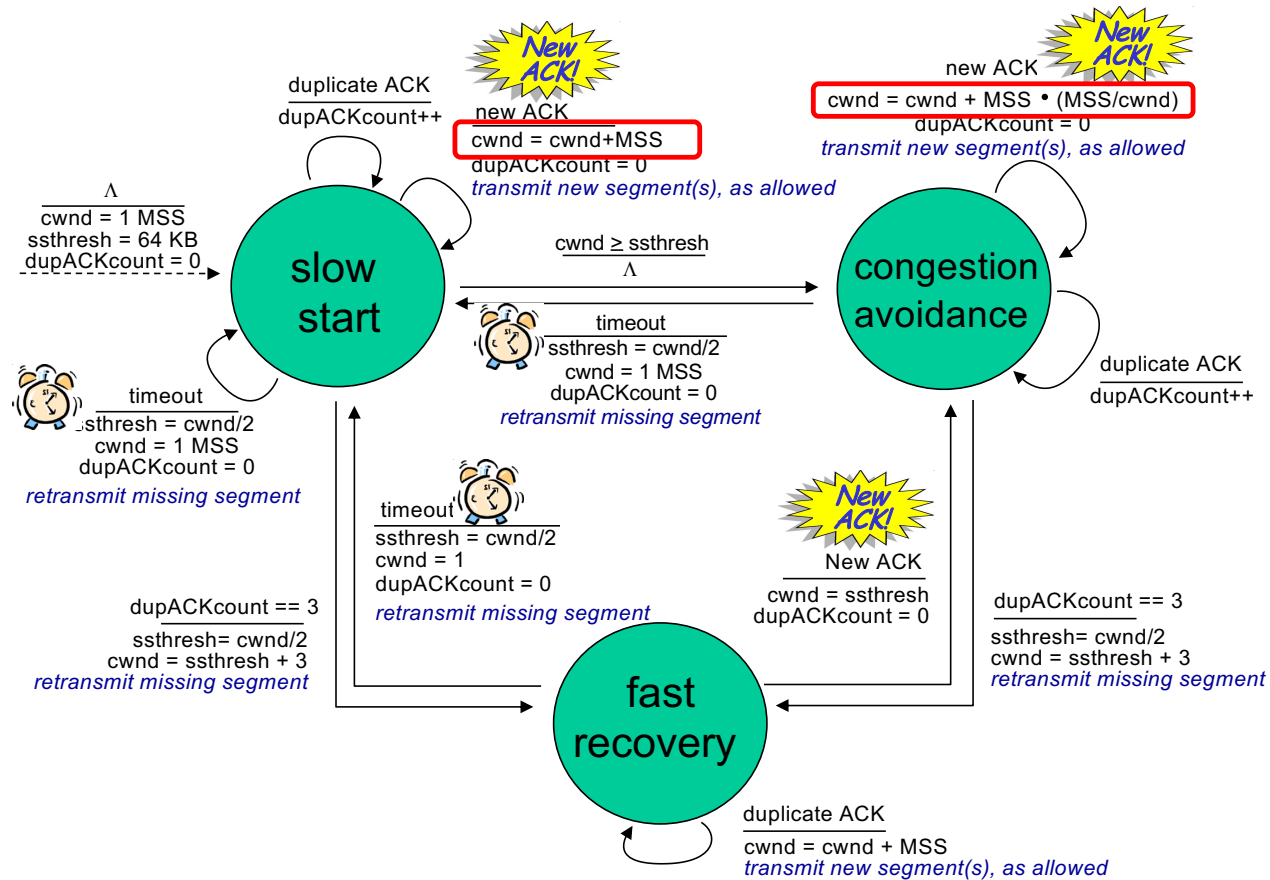
* Check out the online interactive exercises for more examples: http://gaia.cs.umass.edu/kurose_ross/interactive/

Repeat slow start after timeout



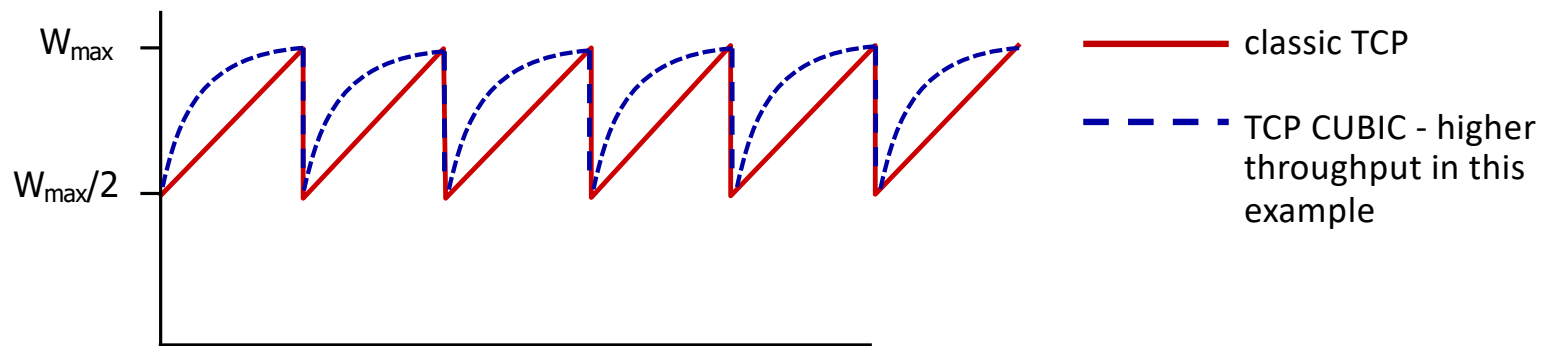
Slow-start restart: : Go back to CWND of MSS, but take advantage of knowing the previous value of CWND

Summary: TCP congestion control



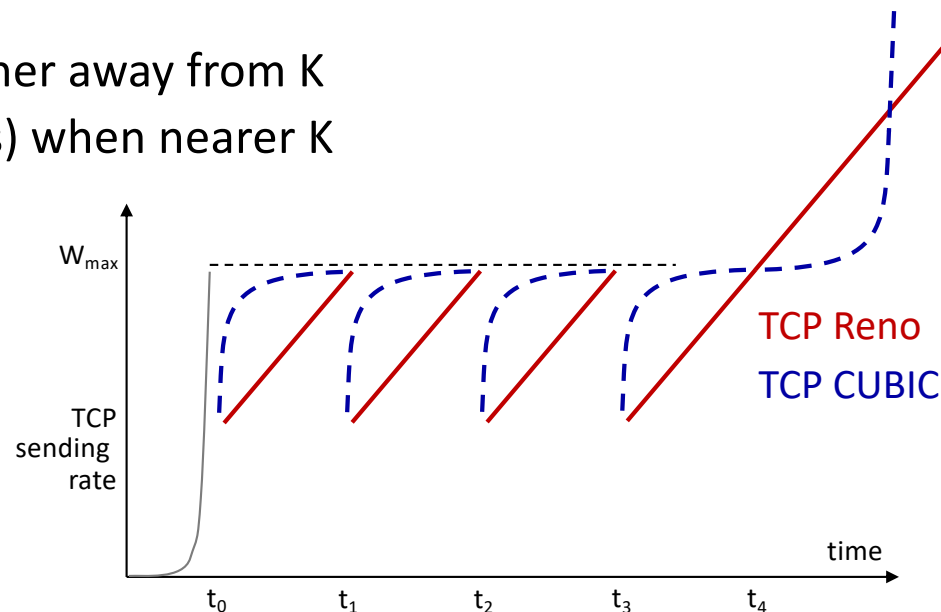
TCP CUBIC

- Is there a better way than AIMD to “probe” for usable bandwidth?
- Insight/intuition:
 - $W_{\max}$: sending rate at which congestion loss was detected
 - congestion state of bottleneck link probably (?) hasn't changed much
 - after cutting rate/window in half on loss, initially ramp to to $W_{\max}$ *faster*, but then approach $W_{\max}$ more *slowly*



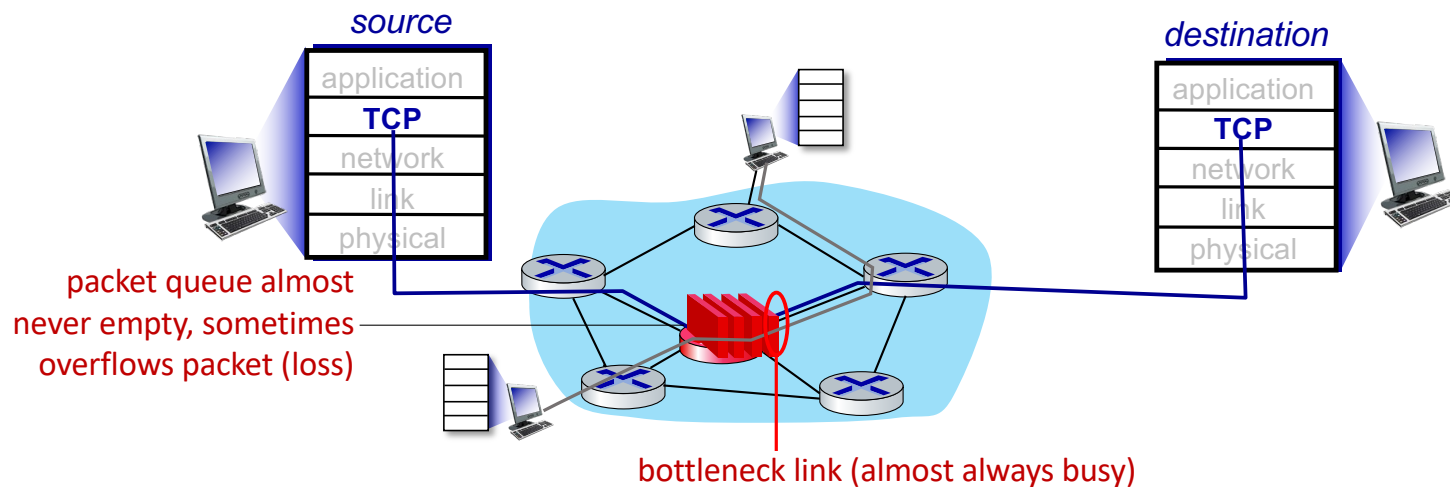
TCP CUBIC

- K: point in time when TCP window size will reach $W_{\max}$
 - K itself is tunable
- increase W as a function of the *cube* of the distance between current time and K
 - larger increases when further away from K
 - smaller increases (cautious) when nearer K
- TCP CUBIC default in Linux, most popular TCP for popular Web servers



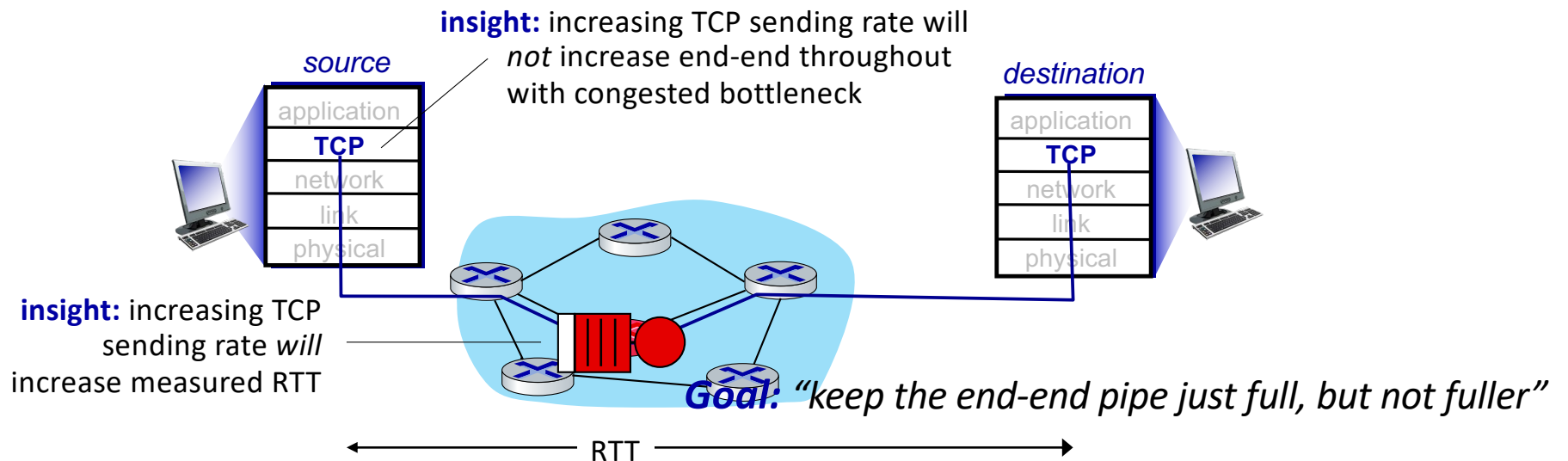
TCP and the congested “bottleneck link”

- TCP (classic, CUBIC) increase TCP’s sending rate until packet loss occurs at some router’s output: the *bottleneck link*



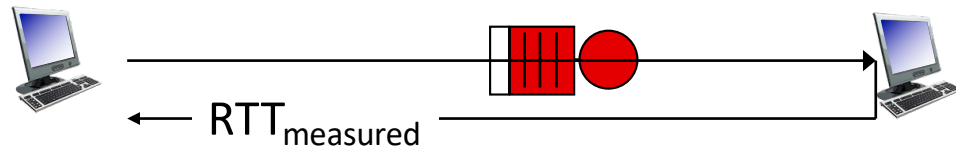
TCP and the congested “bottleneck link”

- TCP (classic, CUBIC) increase TCP’s sending rate until packet loss occurs at some router’s output: the *bottleneck link*
- understanding congestion: useful to focus on congested bottleneck link



Delay-based TCP congestion control

Keeping sender-to-receiver pipe “just full enough, but no fuller”: keep bottleneck link busy transmitting, but avoid high delays/buffering



$$\text{measured throughput} = \frac{\text{\# bytes sent in last RTT interval}}{RTT_{\text{measured}}}$$

Delay-based approach:

- RTT_{min} : minimum observed RTT (uncongested path)
- uncongested throughput with congestion window $cwnd$ is $cwnd/RTT_{\text{min}}$

if measured throughput “very close” to uncongested throughput
increase $cwnd$ linearly /* since path not congested */
else if measured throughput “far below” uncongested throughput
decrease $cwnd$ linearly /* since path is congested */

Is this AIAD?

Delay-based TCP congestion control

- congestion control without inducing/forcing loss
- control is triggered by increased delay/RTT, **not loss events**
- maximizing throughput (“keeping the just pipe full...”) while keeping delay low (“...but not fuller”)
- stable performance: small queue, low delay, minimal loss
- Any issues?
 - detect congestion earlier and reduce cwnd earlier when competing with loss-based TCP → **starvation**

Model-based TCP congestion control

- Build explicit model of the network path and estimate sending rate accordingly
 - Estimate bottleneck bandwidth, propagation delay
- Best for high bandwidth-delay product networks
- BBR (**B**ottleneck **B**andwidth and **R**ound-trip propagation time) deployed on Google's (internal) backbone network

Explicit congestion notification (ECN)

TCP deployments often implement *network-assisted* congestion control:

- two bits in IP header (ToS field) marked *by network router* to indicate congestion
 - *policy* to determine marking chosen by network operator
- congestion indication carried to destination
- destination sets ECN-Echo (ECE) bit on ACK segment to notify sender of congestion
- involves both IP (IP header ECN bit marking) and TCP (TCP header C,E bit marking)

